Design and Evaluation of a Parameterised SIMD Vector Processing Unit on FPGA

Mahir Abdullah, Krishiv Gupta, Abhiman Singh, Gogulamudi Jayadeep, and Ashish Kushwaha

***Abstract*—Signal processing, graphics and machine learning apply the same arithmetic to long arrays of numbers. Single-instruction, multiple-data (SIMD) hardware exploits this regularity: one instruction drives several arithmetic lanes at once. We present the Phase I design of a small SIMD vector processing unit in Verilog whose lane count is a parameter. At eight lanes it holds eight 128-bit vector registers of eight 16-bit elements, replicates a 14-operation lane ALU, broadcasts a scalar operand through per-lane multiplexers and sums the lanes in an adder tree, so that a dot-product step is one instruction. With one lane the same RTL becomes a scalar processor that runs the same programs with the same 48-instruction ISA and timing rules, which gives a like-for-like baseline. An instruction-level golden model predicts every result and the exact cycle count; unit, system, directed, random-program and board testbenches compare the hardware with it, and deliberately injected bugs confirm that the tests catch faults. Every run, including 4,000 random programs, matches the model exactly for 1, 2, 4 and 8 lanes. On five kernels of 256 elements, eight lanes need 7.87–$7.92\times$ fewer clock cycles than one, 1.0–1.7% below ideal, and the shortfall is the serial set-up code predicted by Amdahl's law. Built with every instruction in use and placed and routed on a Lattice ECP5-25F, the eight-lane unit uses 22.2% of the logic cells, and its estimated maximum clock is 29.1 MHz against 43.3 MHz for one lane, so the wall-clock speed-up is $5.3\times$ and throughput rises from 5.6 to 29.9 million elements per second.**

***Index Terms*—Data-level parallelism, FPGA, golden model, hardware verification, SIMD, vector processor, Verilog.**

I. Introduction

A scalar processor spends most of its effort on control: fetching, decoding and sequencing each instruction. When a program adds two arrays, that effort is repeated for every element, although every element needs the same work. A SIMD processor issues one instruction for a group of elements. The control cost is paid once and the arithmetic is replicated across *lanes*. This is *data-level parallelism*, the form of parallelism behind vector supercomputers [1], multimedia instruction-set extensions [2] and graphics processors [3].

Problem statement. The project asks for a small SIMD unit that performs identical arithmetic and logic operations in parallel, built from a vector register file, parallel ALUs, a vector control unit, lane multiplexers, a result combiner and an optional scalar interface. Scalar and SIMD versions of the same workload must be compared, the vector results verified, and cycle counts, throughput, FPGA resources and maximum frequency measured. We answer three questions with measurements:

1) How close to the ideal $L\times$ does an L-lane unit come in clock cycles?
2) What does each extra lane cost in FPGA logic, multipliers and memory?
3) How much does the wider datapath slow the clock, and what wall-clock speed-up remains?

This paper reports Phase I of a two-phase project. Its contributions are:

- a synthesizable SIMD unit in Verilog whose lane count is a parameter, with every block named in the brief (Section III);
- a 48-instruction ISA, an assembler and an instruction-level golden model that predicts results and exact cycle counts (Sections III-B and III-E);
- a layered verification flow with self-checking testbenches, directed corner tests, random programs, a board-level check and mutation tests (Section III-F);
- a scalar-versus-SIMD comparison on five kernels and an FPGA resource and frequency sweep over 1, 2, 4 and 8 lanes (Section IV).

Section II reviews related work. Section III describes the design, its tools and its verification, and Section IV presents the results. Section V discusses them and their limits, and Section VI concludes with the Phase II plan. Fig. 1 gives an overview of the flow that produces every result.

II. Literature Review

Taxonomy and vector machines. Flynn's taxonomy [4] classifies machines by the number of instruction and data streams; SIMD names one instruction stream acting on many data elements. The first commercially successful SIMD designs were vector supercomputers such as the Cray-1 [1], whose vector registers and pipelined functional units let one instruction process up to 64 elements, hiding memory latency and avoiding the issue cost of a scalar loop. Espasa *et al.* [5] review the history of vector architectures and argue that their strengths, few instructions per unit of work and simple control, suit multimedia workloads. Hennessy and Patterson [3] trace the same principles from vector machines to subword SIMD and GPUs.

Subword SIMD in general-purpose processors. Intel MMX [2] introduced *packed* data types: one 64-bit register treated as eight bytes or four 16-bit words, with one instruction operating on all of them. Streaming SIMD Extensions (SSE) [6] widened the registers to 128 bits, the width also

This work forms Phase I of Project 11, SIMD Vector Processing Unit, November 2026. The authors are with Newton School of Technology, Rishihood University.

Code, data and all reported results are available at https://github.com/mahir-m01/SIMD-Vector-Processing-Unit.

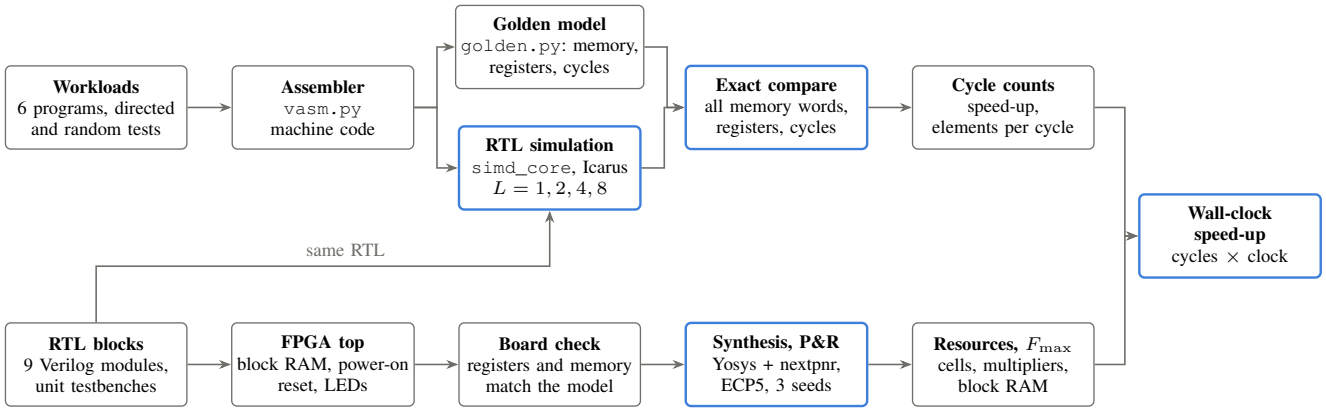


Fig. 1. Phase I flow. Top: each program is assembled once and run on the golden model and on the RTL core at every lane count; memory, registers, instruction count and cycle count must match exactly. Bottom: the same RTL is wrapped with block-RAM memories, checked from power-on in simulation, then placed and routed for resources and maximum clock frequency. Cycle counts and clock frequency together give the wall-clock speed-up.

used by Arm Neon. Two conventions from this line of work are adopted directly here: compare instructions that write all-ones or all-zeros masks, so that conditionals need no branch, and a fixed 128-bit register divided into lanes.

Scalable vector architectures. VIRAM [7] showed that a vector architecture with lanes scales across embedded media workloads more simply than superscalar or VLIW designs, and that a compiler can vectorise most kernels automatically. Asanović [8] analysed lane-based vector microprocessors in depth, and Lee *et al.* [9] found vector cores more efficient than MIMD, subword-SIMD and SIMT designs even on fairly irregular kernels. Two current ISAs leave the vector length to the implementation: the RISC-V vector extension [10] and the Arm Scalable Vector Extension, which allows registers from 128 to 2048 bits under one vector-length-agnostic programming model [11]. Ara [12] implements RISC-V V with identical lanes, each holding a slice of the vector register file, and reaches over 1 GHz in 22-nm technology. The lane replication used in this project follows the same idea at a much smaller scale.

Current SIMD instruction sets. Table I compares this unit with instruction sets in use today. Intel SSE2 and AVX2 [13], Arm Neon [14] and the MIPS SIMD Architecture (MSA) [15] fix the register width in the ISA; RISC-V V lets one binary run on any width through `vsetvli`. Three conventions are adopted here. The 128-bit register and the all-ones compare mask follow SSE, Neon and MSA. The vector–scalar form follows RISC-V V's `.vx` instructions, which broadcast a scalar register inside the instruction. A full sum across lanes follows Neon's `ADDV` and RISC-V V's `vredsum`, whereas x86 `PMADDWD` and MSA `DOTP_S` only add products in pairs; those two keep each product at 32 bits, while `VDOT` keeps its low 16 bits.

Soft vector processors on FPGAs. FPGAs suit lane-replicated designs because the lane count can be matched to the device. VESPA [16] showed a soft vector processor whose lane count and width can be scaled and customised to each application. VIPERS [17] used a soft vector unit as a general-purpose accelerator for a Nios II processor and reported up to 44× speed-up over a Nios II/s with 32 lanes, at 26× its area. VEGAS [18] replaced the vector register file with a scratchpad and introduced fracturable ALUs, and the VectorBlox MXP [19] turned these ideas into a commercial core with 1 to over 128 ALUs. Vicuna [20] is an open-source RISC-V vector coprocessor designed so that its execution time is predictable, which allows tight worst-case timing analysis. Most of these works report speed-up against a separate host processor, so the baseline differs from the vector unit in microarchitecture as well as in width.

TABLE I
SIMD Instruction Sets Compared

ISA	Width (bits)	Regs	Int. elements	Sum across lanes
SSE2	128	8/16	8–64	pairs (`PMADDWD`)
AVX2	256	16	8–64	pairs (`VPMADDWD`)
Neon (A64)	128	32	8–64	full (`ADDV`)
MIPS MSA	128	32	8–64	pairs (`DOTP_S`)
RISC-V V	`VLEN`	32	8–64	full (`vredsum`)
This work	$16L$ (128)	8	16	full, 16-bit products

Performance limits. Amdahl's law [21] bounds the speed-up of any parallel unit by the fraction of work that stays serial; in a vector loop that fraction is the set-up code, the loop control and the final reduction. Gustafson's observation [22] that larger problems shrink the serial fraction explains why speed-up grows with array length.

Verification and tools. Comparing a design against an independent instruction-level model is standard practice in processor verification, and mutation testing [23] judges a test suite by whether it detects deliberately injected faults. Open-source flows make resource and timing results reproducible: Icarus Verilog [24] simulates, Verilator [25] lints, and Yosys [26] with nextpnr [27] synthesise, place and route designs for the Lattice ECP5 using documented timing models.

Gap addressed. Published soft vector processors are large and are usually compared against a different scalar core. This project builds a design small enough for every line to be explained and verified. Its scalar baseline is the *same* RTL with one lane, so any measured gain comes from data-level parallelism alone, and its timing is as predictable as Vicuna's in a far simpler form: the cycle count of every program follows

TABLE II
DESIGN PARAMETERS

Parameter	Value	Meaning
`LANES` (L)	1, 2, 4, 8	lanes in parallel (default 8)
`W`	16	bits per element (signed)
vector registers	$8 \times 16L$ bits	`v0`–`v7`
scalar registers	8×32 bits	`s0`–`s7`, `s0` always 0
`IADDR_W`	8	256 instructions
`DADDR_W`	10	1,024 data words (also on the FPGA)

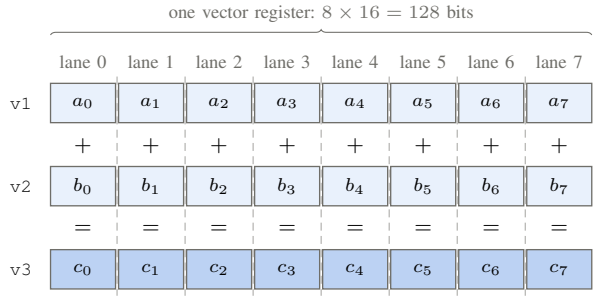


Fig. 2. One SIMD instruction. A 128-bit register holds eight 16-bit elements, one per lane, and `VADD` adds all eight pairs at once. Lanes are independent: no carry crosses a lane boundary (dashed). The scalar build ($L = 1$) runs the same program with one element per register.

from its instruction and load counts.

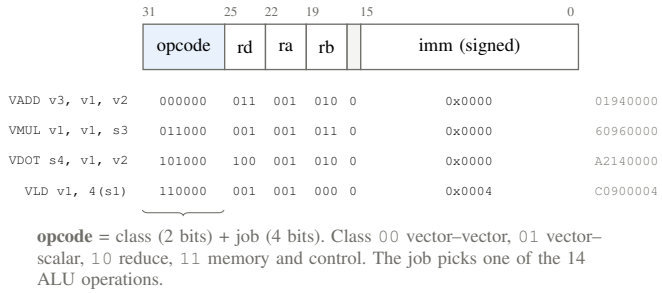
III. PROPOSED SOLUTION

A. Design Goals and Parameters

The design favours clarity over speed. Every instruction except a load completes in one clock cycle, so there are no pipeline hazards and the cycle count of a program is exact. Two parameters set the shape (Table II): the number of lanes L and the element width $W = 16$ bits. With $L = 8$ a vector holds $8 \times 16 = 128$ bits, the width of SSE and Neon (Fig. 2). The memories sit outside the core, so the same RTL is used in simulation and on the FPGA. The RTL is plain Verilog (IEEE 1364-2005 [28]): one module per block, `generate` loops for the lanes and no vendor primitives.

B. Instruction Set

Instructions are 32 bits: a 6-bit opcode, three 3-bit register fields and a signed 16-bit immediate (Fig. 3). The two top opcode bits select one of four classes (Table III); in the first three, the low four bits select one of 14 ALU jobs: `ADD`, `SUB`, `AND`, `OR`, `XOR`, three shifts (`SLL`, `SRL`, `SRA`, by $b \bmod 16$), `MUL` (low 16 bits), `SEQ`, `SLT`, `SLTU`, `MIN` and `MAX`. Fourteen jobs in three forms plus six memory, scalar and control instructions give 48 instructions. Compare jobs write `0xFFFF` for true and 0 for false, so a result can be used directly as a mask with `VAND`. Arithmetic wraps around on overflow ($32767 + 1 = -32768$). Data addresses wrap modulo the memory size and the program counter wraps after 255. Unused opcodes in the three ALU classes write 0 to their destination, and the others do nothing for one cycle, so every bit pattern has a defined effect.



opcode = class (2 bits) + job (4 bits). Class `00` vector–vector, `01` vector–scalar, `10` reduce, `11` memory and control. The job picks one of the 14 ALU operations.

Fig. 3. Instruction format and four encodings produced by the assembler (right: the 32-bit word in hexadecimal). Bit 16 is unused.

TABLE III
INSTRUCTION CLASSES (i RUNS OVER ALL LANES)

Instruction	Opcode	Effect
V*op* `vd, va, vb`	`0x00`–`0x0D`	$v_d[i] = v_a[i] \circ v_b[i]$
V*op* `vd, va, sb`	`0x10`–`0x1D`	$v_d[i] = v_a[i] \circ s_b$ (broadcast)
`VREDop sd, va, vb`	`0x20`–`0x2D`	$s_d = \sum_i (v_a[i] \circ v_b[i])$
`VLD vd, imm(sa)`	`0x30`	$v_d = M[s_a + imm]$, 2 cycles
`VST vb, imm(sa)`	`0x31`	$M[s_a + imm] = v_b$
`ADDI sd, sa, imm`	`0x38`	$s_d = s_a + imm$
`ADD sd, sa, sb`	`0x39`	$s_d = s_a + s_b$
`BNE sa, sb, label`	`0x3C`	branch if $s_a \neq s_b$
`HALT`	`0x3F`	stop

`VDOT` is another name for `VREDMUL`: a dot-product step in one instruction. As with `VMUL`, each product keeps its low 16 bits before the 32-bit sum. The vector-add kernel shows the style of every workload:

```
      ADDI s1, s0, 0        # word index
      ADDI s2, s0, ITER     # passes = N / L
loop: VLD  v1, A(s1)
      VLD  v2, B(s1)
      VADD v3, v1, v2       # L additions at once
      VST  v3, C(s1)
      ADDI s1, s1, 1
      BNE  s1, s2, loop
      HALT
```

The same source runs on every lane count; only the constant `ITER` changes.

C. Microarchitecture

Fig. 4 shows the unit with eight lanes. Following one instruction, `VMUL v3, v1, s2`, through it: the control unit decodes class `01` and job `MUL`; the vector register file reads `v1` while the scalar register file reads `s2`; the lane multiplexers copy the low 16 bits of `s2` into every lane; all eight ALUs multiply at once; and at the next clock edge the combined 128-bit result is written to `v3`. All of this happens within one clock cycle.

Vector control unit. Decodes each instruction into control signals and computes the next program counter. Instruction memory is synchronous, so the unit sends it the address of the *next* instruction one cycle early, including the target of a taken branch; branches therefore cost no extra cycle. A load takes two cycles: the address goes out in the first while the

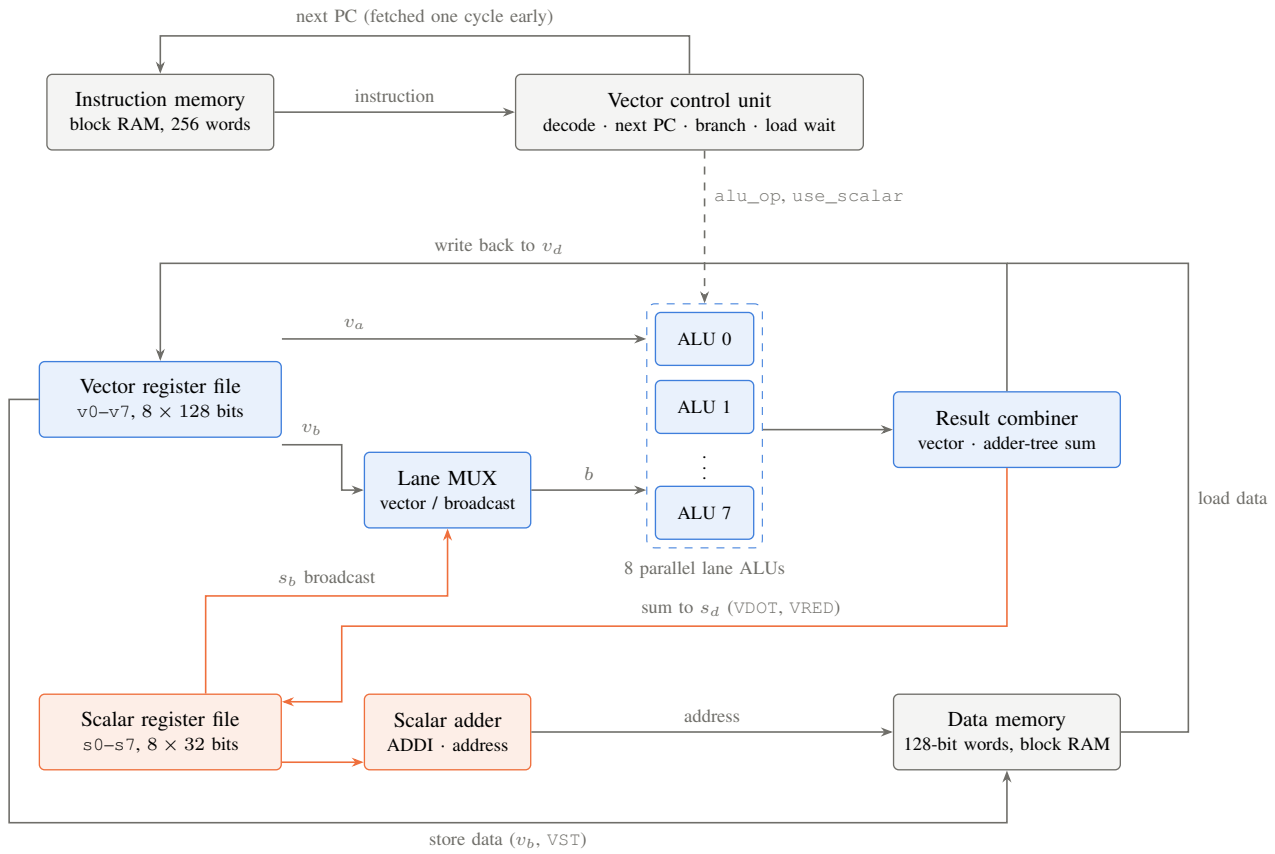


Fig. 4. Block diagram of the SIMD vector processing unit with eight lanes. Blue: vector datapath; orange: scalar side; grey: control and memories; dashed: control signals. The instruction and data memories are synchronous block RAM outside the core.

program counter holds, and the returned data is written in the second.

Vector register file. Eight $16L$-bit registers with two asynchronous read ports and one synchronous write port.

Scalar register file (scalar interface). Eight 32-bit registers, with `s0` hard-wired to zero. They hold addresses, loop counters, broadcast values and reduction results. A third, debug read port exposes any register to the testbench, or to the LEDs on the board.

Lane multiplexers. For each lane, operand B is either that lane's element of v_b or the low 16 bits of s_b copied into every lane.

Parallel ALUs. L copies of a combinational 14-operation lane ALU, created with a `generate` loop and sharing one operation code. Lanes have no connection to each other.

Result combiner. Passes the lane outputs on as one vector for write-back. For reduce instructions it also sign-extends them to 32 bits and adds them in a $\log_2 L$-level adder tree (Fig. 5), so the sum cannot overflow.

FPGA top. Adds block-RAM instruction and data memories loaded at power-up, a power-on reset that holds the core for 15 clocks, a two-flip-flop synchroniser on the reset button, three switches that select a scalar register and eight LEDs that show its low byte.

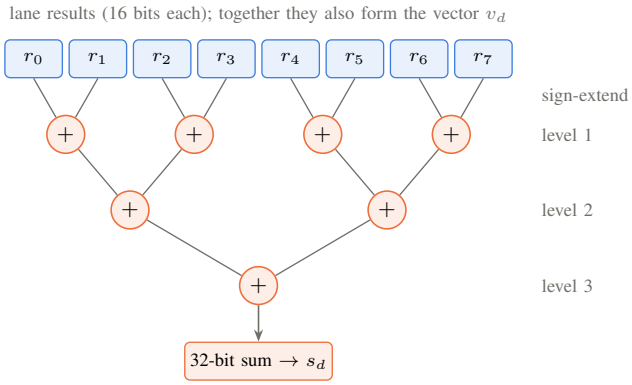


Fig. 5. Result combiner for eight lanes. Reduce instructions widen each lane result to 32 bits and sum them in $\log_2 8 = 3$ levels of adders; vector instructions use the lane results directly as the destination vector.

D. Timing Model

Every instruction except `VLD` completes in one cycle (Fig. 6), so the run time of a program is

$$\text{cycles} = \text{instructions} + \text{loads}. \tag{1}$$

For a kernel with k set-up instructions, a loop body of b instructions containing ℓ loads, and N elements on L lanes,

$$C(L) = k + \frac{N}{L}(b + \ell) + 1, \qquad S(L) = \frac{C(1)}{C(L)} < L, \tag{2}$$

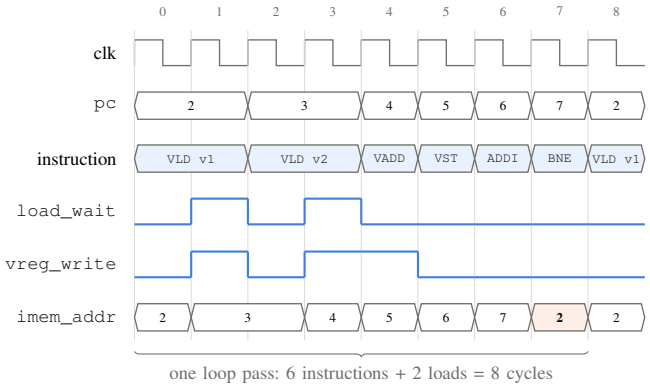


Fig. 6. One pass of the vector-add loop at the cycle level. Each `VLD` holds the program counter for one cycle and writes its register in the second. In cycle 7 the taken `BNE` already sends the loop start (address 2, orange) to instruction memory, so the branch costs nothing.

where the final 1 is `HALT`. The speed-up S approaches L as N grows; the $k+1$ serial instructions are the Amdahl term.

E. Software Tool Flow

Assembler. A two-pass assembler (`vasm.py`) resolves labels in the first pass and encodes in the second. It accepts named constants such as `ITER`, chooses the vector–vector or vector–scalar form from the third operand, and reports errors with their line numbers.

Golden model. An instruction-level simulator (`golden.py`) executes the assembled words with the same wrap-around rules as the hardware and predicts the final data memory, the scalar registers, the instruction count and the cycle count from (1). It shares no code with the RTL.

Workload runner. For each program and lane count, `run_workloads.py` generates input data with a fixed seed, assembles the program with $ITER = N/L$, runs the golden model and the RTL simulation, compares them and writes the cycle counts to a CSV file.

F. Verification Methodology

Verification works in layers (Table IV). Unit testbenches drive each block with corner cases and seeded random stimulus and compare it against reference models written independently of the RTL (for example, shifts as multiplication or division by 2^n); the ALU test also counts the true and false outcomes of each compare and fails on any coverage hole. The system testbench loads a program and its data, runs the core to `HALT`, keeps the clock running for 50 more cycles (no memory write is allowed) and then compares every data-memory word, every scalar register (through the debug port), every vector register and both counters with the golden model. The golden-model result is itself checked against plain arithmetic, such as $C_i = A_i + B_i$, which catches errors in the programs. Directed programs exercise what the benchmarks never do: writes to `s0`, address and program-counter wrap-around, back-to-back loads, a branch into a load, a 32-bit scalar broadcast, every unused opcode, branches on numbers that differ only in bit 16 or bit 31, unused instruction fields filled in, and instructions after `HALT` that must never run. Random programs go further: a fuzzer generates programs with every opcode, random register fields, branches, addresses and memory contents, and each must match the golden model. The board check simulates the FPGA top exactly as built, started only by its power-on reset, and compares its registers and all of its data memory. Finally, mutation tests inject bugs and require the tests to fail. A sixth program, `isa_test`, runs all 42 vector-ALU and reduce forms on random data, so every instruction is exercised on the complete core. Continuous integration repeats the lint and the tests on every change to the repository.

TABLE IV
VERIFICATION PLAN

Testbench	What it checks	Checks
`tb_vec_alu`	14 jobs, 8 lanes: corner-value pairs, every shift, no carry between lanes, 2,000 random vectors per job	225,496
`tb_regfiles`	all registers and ports, write enable, read during write, `s0` = 0	10,044
`tb_lane_combiner`	broadcast and vector modes; sums at extreme values; $L = 1, 2, 4, 8$	$4 \times 11{,}008$
`tb_simd_core`	6 programs: memory, all registers, cycles; no write after `HALT`	24 runs
directed tests	corner-case programs, same testbench	12 runs
random programs	every opcode; random fields, branches, data	4,000
`tb_fpga_top`	board version, `isa_test`, from power-on: registers, memory	4 builds
software tests	encodings, branches, errors, model corners	11

G. Workloads and Experimental Setup

Five kernels (Table V) run on $N = 256$ random 16-bit integers over the full range, drawn with a fixed seed. The plain-arithmetic checks wrap each result to 16 bits as the hardware does (for `VDOT`, each product keeps its low 16 bits). The cycle counts do not depend on the data, because no kernel branches on it. Threshold is a branch-free conditional: a compare builds a mask, an `XOR` with all ones inverts it and an `AND` applies it. Simulation uses Icarus Verilog 13 (12 in continuous integration), lint uses Verilator 5 with all warnings enabled, and synthesis uses Yosys 0.70 for a Lattice ECP5 LFE5U-25F (speed grade 6) with place-and-route in nextpnr-ecp5 0.11. The FPGA builds are loaded with `isa_test`, which uses every instruction: a program that left an instruction class unused would let synthesis remove its logic and make the design look smaller and faster. Each ECP5 build is placed with three seeds, and the median of the maximum frequency $F_{\max}$ estimated by nextpnr's static timing analysis is given. A Yosys estimate for the Xilinx Artix-7 xc7a35t of the Basys-3 board, flattened like the ECP5 build, is also reported.

IV. Prototype Results

A. Verification

All unit tests pass. All 24 system runs (six programs at four lane counts), all 12 directed runs and 4,000 random programs

TABLE V
BENCHMARK KERNELS: SET-UP INSTRUCTIONS k, LOOP BODY b AND LOADS ℓ

Kernel	Operation	k	b	ℓ
vadd	$C_i = A_i + B_i$	2	6	2
saxpy	$Y_i = aX_i + Y_i$	3	7	2
relu	$Y_i = \max(X_i, 0)$	2	5	1
threshold	$Y_i = X_i$ if $X_i \geq T$, else 0	4	7	1
dot	$s = \sum_i A_i B_i$	3	6	2

TABLE VI
MUTATION TESTS: INJECTED BUGS AND THE TEST THAT CAUGHT EACH

Injected bug	Caught by	Result
`SRA` made a logical shift	`tb_vec_alu`	negative inputs
adder tree pairs the wrong lanes	`tb_lane_combiner`	5,002 errors
no wait for load data	`tb_simd_core`	every result
zero- not sign-extension ($L = 1$)	`tb_lane_combiner`	2,507 errors
code after `HALT` keeps running	directed	memory, regs
`BNE` ignores bits 16–31	directed	memory
`VST`/`BNE`/`HALT` write a reg.	`tb_simd_core`	vector regs
FPGA stores to a wrong address	`tb_fpga_top`	stored words

(1,000 per lane count) match the golden model exactly in memory contents, scalar and vector registers, instruction count and cycle count, and the board version matches it in its registers and all 1,024 memory words at every lane count. Verilator lint with all warnings enabled reports nothing for any module; the only two waivers mark bits that are unused on purpose. Every injected mutation was detected (Table VI).

B. Scalar Versus SIMD

Table VII gives the cycle counts. Every one of the 20 kernel counts equals the prediction of (2) exactly. At eight lanes the speed-up is 7.87–7.92×, 1.0–1.7% below the ideal 8× (mean 7.89×), and halving the lane count halves the gain (Fig. 7, left and centre). The shortfall is the two to four set-up instructions and `HALT`, which run once whatever the lane count, so it grows with the ratio of serial instructions to loop work: threshold, with four set-up instructions, loses most (1.7%), and vadd, with two set-up instructions and eight cycles per pass, loses least (1.0%).

Because (2) matches every run, it predicts the speed-up at other array sizes (Fig. 7, right). For vadd at eight lanes it is 6.1× at $N = 8$, 7.9× at $N = 256$ and 7.99× at $N = 4096$ (sizes above 341 elements do not fit the scalar build's memory, so they are predicted, not run): the serial part is fixed, so longer arrays hide it, as Gustafson [22] observed for scaled problems. At $N = 256$ eight lanes complete 0.88 (saxpy) to 1.31 (relu) elements per clock cycle, against 0.11 to 0.17 for one lane.

C. FPGA Resources and Clock Frequency

Table VIII and Fig. 8 give the synthesis results. Each lane adds one hardware multiplier and its own ALU, about 550 logic cells per added lane on average from one to eight lanes. Shared logic (control, scalar registers, memories) is paid once, so eight lanes need 3.5× the logic of one, not 8×. Yosys

TABLE VII
CLOCK CYCLES FOR $N = 256$ ELEMENTS

Kernel	L=1	L=2	L=4	L=8	Speed-up
vadd	2051	1027	515	259	7.92×
saxpy	2308	1156	580	292	7.90×
relu	1539	771	387	195	7.89×
threshold	2053	1029	517	261	7.87×
dot	2052	1028	516	260	7.89×

TABLE VIII
FPGA RESULTS. ECP5-25F AFTER PLACE-AND-ROUTE ($F_{\max}$: MEDIAN AND RANGE OF THREE SEEDS); ARTIX-7 XC7A35T YOSYS ESTIMATE

	ECP5-25F					Artix-7	
L	Cells	FF	Mult.	BRAM	$F_{\max}$ (MHz)	LUTs	DSP
1	1,555 (6.4%)	16	1	2	43.3 (42.0–44.3)	548	1
2	1,943 (8.0%)	16	2	3	36.2 (36.1–37.2)	864	2
4	3,720 (15.3%)	16	4	5	32.9 (32.7–33.7)	1,483	4
8	5,396 (22.2%)	16	8	9	29.1 (28.9–29.8)	2,731	8

maps both register files to distributed LUT RAM, so the ECP5 build uses only 16 flip-flops at every lane count (the program counter, the load and halt flags and the reset logic); block RAM grows from two to nine blocks because the data words widen with L. On the Artix-7, Yosys estimates 2,731 LUTs (13%) and eight DSP slices, one per lane, for eight lanes.

The estimated maximum frequency falls from 43.3 MHz at one lane to 29.1 MHz at eight, 33% lower; the spread between placement seeds is at most 2.3 MHz. Because the core executes each instruction in a single cycle, its critical path crosses the whole datapath. At eight lanes nextpnr reports it from the instruction block RAM through the scalar and vector register reads, a lane ALU (the multiplier in two of the three placements), the result selection and the adder tree to the scalar register write. Wider designs also spread over more of the device, which lengthens the routing.

D. Throughput and Area Efficiency

Combining cycles and clock, the eight-lane unit finishes a kernel $7.89 \times 29.1/43.3 = 5.3\times$ sooner than the scalar build, against 1.7× for two lanes and 3.0× for four (Fig. 7, centre). Throughput rises from 5.6 to 9.4, 17.0 and 29.9 million elements per second (Fig. 8, right). Per 1,000 logic cells it grows from 3.6 million elements per second at one lane to 5.5 at eight: eight lanes do 1.5 times as much work per cell as one, because the shared logic is spread over more lanes.

V. DISCUSSION AND LIMITATIONS

Near-ideal scaling in cycles. The lanes never interact, the core has no hazards, and every kernel's serial part is three to five instructions, so the cycle speed-up is within 1.7% of ideal and is predicted exactly by (2). The unit therefore demonstrates the main promise of SIMD: the work done per instruction grows with the number of lanes while the control cost does not.

The clock pays for the width. Single-cycle execution makes the design simple to verify and its timing exact, but

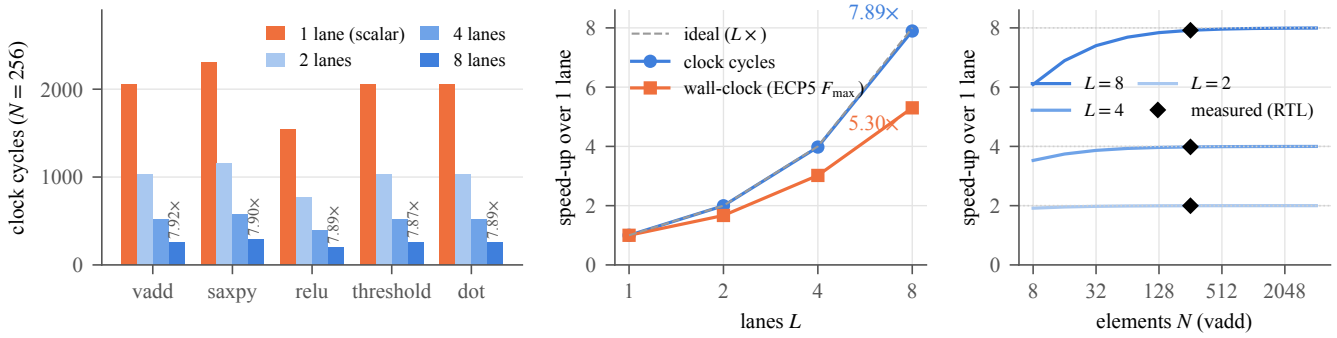


Fig. 7. Scalar versus SIMD. Left: clock cycles per kernel at 1, 2, 4 and 8 lanes, with the eight-lane speed-up. Centre: mean speed-up over the five kernels in cycles and in wall-clock time (with each build's ECP5 $F_{\max}$); dashed: ideal. Right: vadd speed-up against N from (2); diamonds: RTL measurements at $N = 256$.

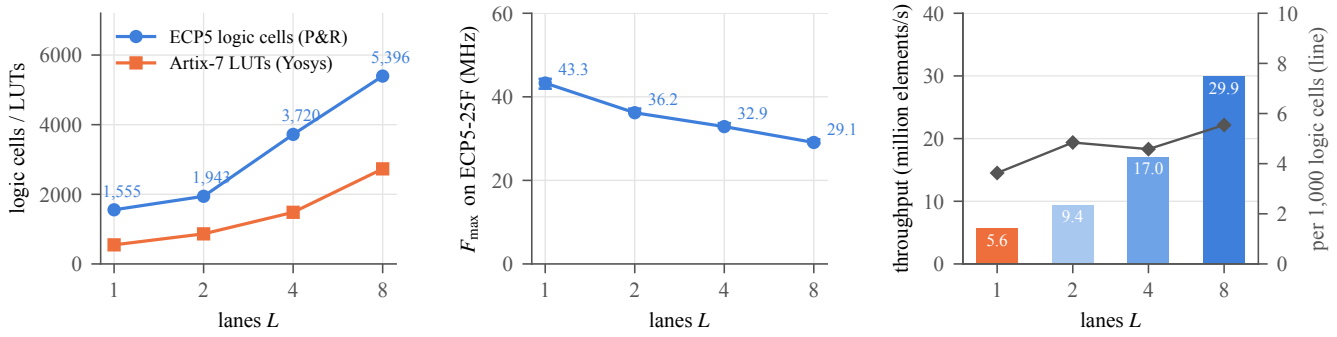


Fig. 8. FPGA scaling with every instruction in use. Left: logic on the ECP5-25F after place-and-route and on the Artix-7 (Yosys estimate). Centre: ECP5 $F_{\max}$, median of three seeds (bars: range). Right: throughput, mean elements per cycle times $F_{\max}$ (bars), and the same per 1,000 logic cells (line).

puts the multiplier and the adder tree on the same path as the instruction fetch and the register write. A third of the clock is lost between one and eight lanes, so a wider single-cycle datapath delivers less than its cycle count suggests; pipelining is needed to keep the clock rate.

The loop, not the ALU, sets the pace. In one pass of vadd at eight lanes only one of the eight cycles does arithmetic; two loads take four cycles, and the store and the loop control take three (Fig. 6). The ALUs are idle most of the time. Memory bandwidth is not the limit, since the block RAM could deliver a 128-bit word every cycle; the limit is issue: one instruction per cycle, loads that take two cycles and are not overlapped, and the loop control. Loop unrolling, auto-incrementing addresses or pipelined loads would raise throughput without adding lanes.

Limitations and threats to validity.

- The design is not pipelined, so the clock frequencies are those of a single-cycle datapath and understate what the lane array could reach.
- Elements are 16-bit integers only, with wrap-around and no saturation, and there is no masked (predicated) execution; conditionals use `AND` masks.
- `VDOT` keeps only the low 16 bits of each product, so its inputs must be small enough for the products to fit, as in the dot-product test; a widening multiply would remove this limit.
- A program must be reassembled for each lane count, because the loop count N/L is a constant; there is no `vsetvli`-style instruction.
- The FPGA results are for one device and speed grade, and $F_{\max}$ is nextpnr's static-timing estimate without pin constraints. Artix-7 figures are Yosys estimates without timing, and the Vivado flow for the Basys-3 board is written but not yet run. The board check simulates the FPGA top; the design has not yet run on a physical board.
- The kernels are short and regular. No power or energy was measured.

VI. Conclusion and Phase II Plan

The Phase I prototype is complete and verified. An eight-lane, 128-bit SIMD unit executes a 48-instruction ISA, matches its golden model cycle for cycle at every lane count, and reaches a 7.9× cycle speed-up and a 5.3× wall-clock speed-up over its own scalar configuration on a mid-range FPGA, at 22% of the logic. The results show where the remaining performance lies: in the clock, which a pipeline would recover, and in the loop overhead around the arithmetic. Phase II will therefore (i) pipeline the core to raise $F_{\max}$ and recover wall-clock speed-up; (ii) measure exact Artix-7 resources and timing in Vivado and run the design on a Basys-3 board; (iii) add predicated (masked) execution and saturating

arithmetic; (iv) reduce loop overhead with unrolling and auto-incrementing addresses, and measure speed-up against problem size on the hardware; and (v) compare estimated power per operation for the scalar and SIMD builds.

Code and Data Availability

All RTL, testbenches, programs, scripts, results and figures are in the repository named on the first page. One command, `make test`, runs the lint, the unit testbenches, the software tests, the system and directed runs, 100 random programs and the board check; `make fuzz` runs 4,000 random programs; `make perf` and `make syn` regenerate the cycle counts and the FPGA results, and `make report` rebuilds the figures and this paper from them.

Individual Contributions

Mahir Abdullah (lead): the architecture, instruction format and block interfaces that the team built against; the vector control unit with early instruction fetch and two-cycle load; core integration with performance counters; the instruction-level golden model, the workload runner, the directed tests and the random-program fuzzer; the system testbench; the repository, continuous integration and documentation; and this report.

Krishiv Gupta: the 14-operation lane ALU and its `generate`-based array, including signed and unsigned compares and arithmetic shifts, with its 225,496-check testbench, which also confirms that no carry crosses a lane; the FPGA top with block-RAM memories and power-on reset; the board check; the ECP5 and Artix-7 synthesis sweep, the Vivado script and Basys-3 constraints; and the resource and $F_{\max}$ analysis.

Abhiman Singh: the vector and scalar register files, with two read ports and one write port each, a hard-wired zero register and the debug port that forms the scalar interface; and their testbench, which covers read-during-write behaviour and random cycles against a shadow model.

Gogulamudi Jayadeep: the lane multiplexers and the result combiner with its $\log_2 L$ adder tree, verified for every lane count; and the scalar-versus-SIMD analysis: cycle counts for 1, 2, 4 and 8 lanes, speed-up, elements per cycle and the figures.

Ashish Kushwaha: the two-pass assembler with labels, constants and line-numbered error messages, and its tests; and the six workload programs, including the branch-free threshold kernel and the instruction-coverage program.

Acknowledgment

The authors thank the developers of Icarus Verilog, Verilator, Yosys and nextpnr, whose open tools make every result in this report reproducible.

References

[1] R. M. Russell, "The CRAY-1 computer system," *Commun. ACM*, vol. 21, no. 1, pp. 63–72, Jan. 1978.

[2] A. Peleg and U. Weiser, "MMX technology extension to the Intel architecture," *IEEE Micro*, vol. 16, no. 4, pp. 42–50, Aug. 1996, doi: 10.1109/40.526924.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017, ch. 4.

[4] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Trans. Comput.*, vol. C-21, no. 9, pp. 948–960, Sep. 1972.

[5] R. Espasa, M. Valero, and J. E. Smith, "Vector architectures: Past, present and future," in *Proc. 12th Int. Conf. Supercomputing (ICS)*, 1998, pp. 425–432.

[6] S. K. Raman, V. Pentkovski, and J. Keshava, "Implementing streaming SIMD extensions on the Pentium III processor," *IEEE Micro*, vol. 20, no. 4, pp. 47–57, Jul. 2000, doi: 10.1109/40.865866.

[7] C. E. Kozyrakis and D. A. Patterson, "Scalable vector processors for embedded systems," *IEEE Micro*, vol. 23, no. 6, pp. 36–45, Nov. 2003, doi: 10.1109/MM.2003.1261385.

[8] K. Asanović, "Vector microprocessors," Ph.D. dissertation, Univ. California, Berkeley, CA, USA, 1998.

[9] Y. Lee, R. Avizienis, A. Bishara, R. Xia, D. Lockhart, C. Batten, and K. Asanović, "Exploring the tradeoffs between programmability and efficiency in data-parallel accelerators," in *Proc. 38th Int. Symp. Comput. Archit. (ISCA)*, 2011, pp. 129–140.

[10] RISC-V International, "The RISC-V 'V' vector extension," version 1.0, 2021. [Online]. Available: https://github.com/riscv/riscv-v-spec

[11] N. Stephens *et al.*, "The ARM scalable vector extension," *IEEE Micro*, vol. 37, no. 2, pp. 26–39, Mar. 2017, doi: 10.1109/MM.2017.35.

[12] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multiprecision floating-point support in 22-nm FD-SOI," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 2, pp. 530–543, Feb. 2020.

[13] Intel Corp., *Intel 64 and IA-32 Architectures Software Developer's Manual*, vol. 2. [Online]. Available: https://www.intel.com/sdm. Accessed: Nov. 6, 2026.

[14] Arm Ltd., *Arm Architecture Reference Manual for A-profile Architecture*. [Online]. Available: https://developer.arm.com/documentation/ddi0487/latest. Accessed: Nov. 6, 2026.

[15] MIPS Technologies, "MIPS SIMD Architecture," white paper, rev. 1.00, 2012.

[16] P. Yiannacouras, J. G. Steffan, and J. Rose, "VESPA: Portable, scalable, and flexible FPGA-based vector processors," in *Proc. Int. Conf. Compilers, Archit. Synth. Embedded Syst. (CASES)*, 2008, pp. 61–70.

[17] J. Yu, C. Eagleston, C. H.-Y. Chou, M. Perreault, and G. Lemieux, "Vector processing as a soft processor accelerator," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 2, no. 2, pp. 12:1–12:34, 2009, doi: 10.1145/1534916.1534922.

[18] C. H. Chou, A. Severance, A. D. Brant, Z. Liu, S. Sant, and G. G. F. Lemieux, "VEGAS: Soft vector processor with scratchpad memory," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, 2011, pp. 15–24.

[19] A. Severance and G. G. F. Lemieux, "Embedded supercomputing in FPGAs with the VectorBlox MXP matrix processor," in *Proc. Int. Conf. Hardware/Software Codesign Syst. Synth. (CODES+ISSS)*, 2013, pp. 1–10, doi: 10.1109/CODES-ISSS.2013.6658993.

[20] M. Platzer and P. Puschner, "Vicuna: A timing-predictable RISC-V vector coprocessor for scalable parallel computation," in *Proc. 33rd Euromicro Conf. Real-Time Syst. (ECRTS)*, ser. LIPIcs, vol. 196, 2021, pp. 1:1–1:18, doi: 10.4230/LIPIcs.ECRTS.2021.1.

[21] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.

[22] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.

[23] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, Apr. 1978.

[24] S. Williams, "Icarus Verilog," 2026. [Online]. Available: https://steveicarus.github.io/iverilog/

[25] W. Snyder, "Verilator," 2026. [Online]. Available: https://www.veripool.org/verilator/

[26] C. Wolf, "Yosys Open SYnthesis Suite." [Online]. Available: https://yosyshq.net/yosys/

[27] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanović, "Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs," in *Proc. IEEE 27th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, 2019, arXiv:1903.10407.

[28] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, 2006.